

A Proposal to Add `split/join` of `string/string_view` to the Standard Library

I. Motivation

Split a string in multiple string based on a separator and the reverse operation to aggregate a collection of string with separator are quite common operations , but there's no standardized easy to use solutions in the existing `std::basic_string` and the proposed `std::basic_string_view` class.

```
split("C++/is/fun", "/") => ["C++", "is", "fun"]
join(["C++", "is", "fun"], "/") => "C++/is/fun"
```

The purpose of this simple proposal is to cover this miss.

We also propose solutions to easily handle case conversion

Theses features are available in the standard string class of the following languages : D, Python, Java, C#, Go, Rust.

II. Impact On the Standard

This proposal is a pure library extension. It does not require changes in the language core itself.

It does require adding new method to `std::basic_string` class (or not if implemented only in `std::basic_string_view`).

Or just a function add in algorithms if this option is preferred.

It has been implemented in standard C++.

III. Design Decisions

Several options have been discussed in this discussion [\[1\]](#), bellows a summary of the various discussed options. As several alternative has been discussed, we let the committee choose which option is privileged.

3.1 Discussion on `split`

3.1.1 `splits` & `splitsv` method

Probably the simplest option is to add method to `std::basic_string` and `std::basic_string_view`.

Example on `std::basic_string_view` (`std::basic_string` is quite the same)

```
vector<basic_string<CharT, Traits> > splits(const basic_string_view<CharT, Traits> &Separator) const
vector<basic_string_view<CharT, Traits> > splitsv(const basic_string_view<CharT, Traits> &Separator) const
```

The purpose of theses method is to return a vector of string or of `string_view`.

It's decided to hardcode the vector choice to have a call as simple possible.

```
auto MyResult= "my, csv, line"s.split(",");
```

`s` and `sv` suffixes are derived from the normalized literal suffixes.

`splitsv` has the advantage to be efficient in terms of CPU (no copy to do) and RAM (No memory to allocate for substring, just for the vector).

`splits` is useful if `splitsv` can't be used. For instance, it's needed if you try to split a temporary object.

Move semantic is used to optimize the returned vector.

Example of implementation in [\[2\]](#) (could be optimized with an initial parsing to reserve the vector)

3.1.2 `basic_string_view` implementation only

Several options presented here are method in both `std::basic_string` and `std::basic_string_view`. It could make sense to implement them only in `std::basic_string_view` for several reasons :

- The implementation is similar for both
- string can easily (and with a very small cost) be converted on `string_view`
- `basic_string_view` is a new class then it's probably simpler to amend it to integrate theses features in C++ 17. Could be back ported later on `std::basic_string` if needed.

3.1.3 `splitf` method

`splitf` method will split the input string and call an unary functor with a `std::basic_string_view` as a parameter

```
template <class F>
void splitf(const basic_string_view &Separator, F functor) const
```

Some person does not wish to have a container returned to avoid its memory allocation, `splitf` is one possible method to address this concern.

Some person wishes to execute a processing on each value, `splitf` is a more direct way to address this request than iterate on `splitsv` result with `for_each`.

The transmitted string view allows compute the position of the substring in the initial string, it was highlighted as a potential need.

Example of usage displaying substring, initial position and length

```
strsv.splitf(" ", [&](const string_view &s) {
    cout << s << "    ", Pos="<<(s.data() - strsv.data())<<"    , Len="<<s.length()<< endl;
});
```

Example of implementation in [\[2\]](#)

3.1.4 splitc method

splitc method will split the input string and push substring in the container passed as output parameter.

```
template <class T>
void splitc(const basic_string_view<CharT, Traits> &Separator, T &Result) const
```

Some person does not wish to use a vector container, this option allows transmit a wide range of containers to address this concern. This option also allows feed a container with another string type in it. Can be done in this way if it can be built from a string_view.

Example of usage

```
vector<string_view> vector5;
strsv.splitc(" ", vector5);
vector<string> vector6;
strsv.splitc(" ", vector6);
```

Example of implementation in [\[2\]](#)

3.1.5 Unify name for split methods

Instead of using suffixes to select the version, it would be nice to have an automatic selection of the right split version.

splits& splitsv have only 1 parameter and can easily be differentiated from splitf & splitc.

splits& splitsv will be selected by parametrizing the return type.

```
str.split<string>(" "); ==> Returns vector<string>
str.split<string_view>(" "); ==> Returns vector<string_view>
```

It's not very user-friendly, but if we set a default type it can be better. I selected basic_string_view as default type as it's the most interesting in most cases and the longer to type (string_view=11 char).

When other type is needed (i.e. string), it can be specified, it can also be another string type if convertible from string_view

```
template<class StringType=basic_string_view<CharT, Traits>, class SeparatorType >
vector<StringType > split(const SeparatorType &Separator) const
```

```
str.split<string>(" "); ==> Returns vector<string>
str.split(" "); ==> Returns vector<string_view>
```

An alternate solution was proposed by Jakob Riedle with a specialization based on the fact invoker is a lvalue or a rvalue.

This solution has the advantage it automatically discards the usage of string_view on temporary values, but it makes it a bit more complex if you really want split as a copy in a vector<string>, it can, however, be done using splitc(MyStringVector). I've tested it successfully with this prototype on GCC&VS2017

```
template<class SeparatorType >
vector<basic_string_view > split(const SeparatorType &Separator) const &;
template<class SeparatorType >
vector<basic_string > split(const SeparatorType &Separator) const &&;
```

To make the difference between splitf & splitc version, enable_if & is_callable may probably be used. I was unable to implement it (I left splitc as is).

Example of implementation in [\[6\]](#)

3.1.6 Special values

If the separator is an empty string "" or the separator char is 0 then split methods will split on every char. "abc"=>["a", "b", "c"]

If the input string is an empty string "" then split methods will return an empty container or never call the callback function.

3.1.7 split by single char

It was highlighted that split by a single char can be optimized regarding splitting by a string. Then it was suggested an overload for single char Separator.

A counter-argument was this optimization can (and should) be detected at runtime or by a non-standardized overload added by the implementor. But why spend time to check if it can be determined at compilation.

For information the string::find method as standardized the char overload too.

It was decided to add the char overload as it also avoid the creation of the temporary string_view object.

The implementors can decide to forward it to the string_view version if it don't want to optimize this case.

Specify it as part of the standard is an incitation to optimize this case as it's probably the most common case.

Example of implementation in [\[2\]](#)

3.1.8 split by regexp

It was highlighted that split by a regexp can be useful, it can also be a way to implement the split by a set of separators, then an overload of split with regexp could make sense.

It may introduce a dependency on regexp which is perhaps not a good idea.

But it could be implemented by a regexp_split function.

Perhaps it's not an issue to have a dependency with another standard class. In this case, an overload on member-function may make sense.

This option is part of the proposed text.

Example of implementation in [\[2\]](#)

3.1.9 Alternate option : split by splitter object

It N3593 Proposal [\[5\]](#) there's the idea of using a template delimiter object which must implement a find method.

The proposal suggests providing 4 built-in classes for char, string, any_of_string, regex.

It's a smart idea as it allows reduce the number of overload and provide the possibility to extend the concept.

However I think that the benefit provided by reduction of overload is lost by the introduction of 4 new classes. And the interest of adding a new kind of splitter is reduced.
Then I consider this solution is a bit more complex for a reduced benefit.

3.1.10 Alternate option : split as non-member function : string_split

string_split function algorithm may replace member function as it doesn't require special access to the class.

Pros are :

- * avoid implementing it in both classes.

It's true, methods have to be declared on both classes, however, in basic_string it can be a generic forwarder (below examples used for unified split).

```
// SPLIT Version replacing splits & splitsv for any separator
template, class SeparatorType >
vector<StringType > split(const SeparatorType &Separator) const
{
    basic_string_view17<CharT, Traits> sv(this->c_str(),this->size());
    return sv.split<StringType>(Separator);
}

// SPLIT Version replacing splitf & splitc for any separator
template<class SeparatorType , class TargetType>
void split(const SeparatorType &Separator, TargetType Target) const
{
    basic_string_view17<CharT, Traits> sv(this->c_str(), this->size());
    return sv.split(Separator, Target);
}
```

- * It can be used with alternate string class, however, it will probably require they have a standardized way to extract substring by having a substr method (it's not the case of qstring, Cstring, AnsiString) making this argument less valuable

- * It may be simpler to use with char*

```
const char *MyCharStr="My char* to split";
```

```
vector4 = string_split(MyCharStr, " ");
```

could be written instead of

```
vector4 = string_view(MyCharStr).split(" ");
```

However during my tests [\[6\]](#) I wasn't able to make char* works with this definition (CharT deduction impossible)

```
template <class CharT,class Traits = std::char_traits<CharT>,
        class StringType = basic_string_view<CharT, Traits> >
vector<StringType > string_split(const basic_string_view17<CharT, Traits> &InputStr, const basic_string_view<CharT, Traits> &Separator)
```

But was possible if I skip the CharT with this definition instead (but it will require to define a function for each possible CharT that is opposed to the 1st Pros argument)

```
template <class StringType = string_view >
vector<StringType > string_split(const string_view17 &InputStr, const string_view &Separator)
```

Cons :

- * why 'find*','substr','copy' can be a member and 'split' should not? It looks to me more homogeneous.

- * I consider using member function is more Object Oriented Programming.

```
mystring.split(',') //seems to me more OOP
string_split(mystring,',') //seems to me more C/Procedural
```

- * Introduction of a 'global_name' string_split which can enter in a collision with a user function. It's supposed to be protected by namespace but usage of using namespace std; is quite common.

3.1.11 Alternate option : string_split function returning a range

string_split function algorithm will return a range and can potentially replace all the previously discussed method

Example of usage

```
string MyStr("my, csv, line")
vector<string> MyResult(split(MyStr, ","));
// Efficient conversion from string to string_view may require an explicit initial casting
vector<string_view> MyResult(split(string_view(MyStr), ","));
// splitf replacement
std::for_each(split(string_view(MyStr), ","), callback);
```

Example of implementation in [\[3\]&\[4\]](#)

It looks at a smart option, however, some cons on it :

- * Temporary object issues (see below)

- * Some notation may be heavier than with methods options

The Temporary object issue

During the evaluation of the prototype it was clearly appears there's a problem with temporary objects.

```
string string_toLower(string Str)
{
    std::transform(Str.begin(), Str.end(), Str.begin(), ::tolower);
    return Str;
}

int main()
{
    for(auto x : split_string(vstr1,regex("\\s")))    <== (1)
        . . . . .
    for(auto x : split_string(string_toLower("C++ Is Very Fun!!!")," "))    <== (2)
        . . . . .
}
```

Code (1) will **not** work because split_string use a reference to the regex but has this one doesn't exist anymore when split_string returns, the returned iterator got a problem because the regex object doesn't exist anymore.

It can easily be fixed by copying the regex in the iterator, but it's not cost free. Problem is potentially the same with the string separator but has it should be a literal it should be OK in most times.

Code (2) will **not** work either and it's a bigger problem. It's for the same reason, when we use the returned iterator, the lowered string doesn't exist anymore. A workaround could be to systematically make a copy of the inputString but it doesn't make sense from a memory and a performance perspective. It will also invalidate returned string_view in several other usage as the returned string view will not be on the initial object but on the saved copy.

A complementary test done using split action of range-v3 library seems to confirm this fact :

```
vector<string> vect1= string_toLower(str8) | view::split(',');
```

This instruction display this assertion :

```
error: static assertion failed: You can't pipe an rvalue container into a view. First, save the container into a named variable, and then pipe it to the view.
```

Which let think that, by design, range cannot handle temporary values.

Solutions Comparison

Since the first answer in the proposal thread on google groups, there's a debate between methods vs function. Bellow a factual comparison on some use case.
Note : Other options like unified name split or non member function can be consider but they are pretty close of method option.

Usage	Range function Option	Method Option	Remarks
Split in vector of string_view	vector<string_view> vec=string_split(str,"");	auto vec=str.splitsv(",");	Function solution allocate 1 extra split_range_string and 3 extra iterators (first_past_last_the internal of for)
Split in vector of string	vector<string> vec=string_split(str,"");	auto vec=str.splits(",");	Function solution allocate 1 extra split_range_string and 3 extra iterators (first_past_last_the internal of for) +creation of intermediate string_view
Split in list of string_view	list<string_view> lst=string_split(str,"");	list<string_view> lst;str.splitc(", ",lst);	Function solution use the same notation for every container. Method solution requires a different coding for other containers but is more efficient as it doesn't require extra object creation.
Split over a function	for_each(string_split(str," "),MyFunction);	str.splitf(" ",MyFunction);	Function solution allocate extra objects
Split over a loop (a)	for(s :string_split(str," "))	for(s :str.splitsv(", "))	Methods solution allocate an extra vector of string_view
Split over a loop (b)	for(s :string_split(str," "))	strsv.splitf(" ", [&](const string_view &s) { cout << s << endl; });	Methods solution doesn't allocate the extra vector and no extra object like function solution but the notation is quite less conventional
Temporary object	vec=string_split(GetTmpObject(),"");	vec=GetTmpObject().splits(",");	Function solution : Doesn't work (see Temporary object issue paragraph) Methods solution : works fine
From alternate string class (Must be convertible from string_view)	vec=string_split(MyOtherString,"");	vec=string_view(MyOtherString).splitsv(",");	both options use in fact the string_view implementation. string_split is not instanciated for the alternate string class.

3.1.12 comparison with range::split

C++ 17 will bring a new feature named range that will allow simplify operations on several stuff, thanks to Eric Niebler for his strong commitment on that. Range-V3 (ancestor of the proposal) include a split action allowing split containers by an element, a range or a predicate. This solution would fit the need but with some restriction I think. Ranges are designed to handle general containers with general purpose content, at the opposite this proposal is string oriented and integrate easy to use features. Split by a single char with range is easy to do and can be a reasonable alternative : vector<string> vect1= str8 | view::split(','); Use range to split by a substring or a regex will be significantly more complex and require significant pieces of technical code. Take advantage of string_view is possible with range but it requires an explicit cast of the input string then I consider it should be the default case. As described in previous paragraph (range function) range usage cause creation of several intermediate objects (the range, iterators), if we consider the case of parsing a 100 000 line CSV file of 10 columns where each line is handled using the random access operator on the application. I've the feeling that using the splitc variant on a vector will be simpler to write and cause significantly less allocation of internal objects.

3.2 Discussion on join

3.2.1 join static method

join static method will join a list of input string transmitted on a iterable container and add a delimiter between each value

```
template<class T,class U>
static basic_string<CharT, Traits, Allocator> join(T &InputStringList, U Separator)
```

Example of usage (simple but so useful)

```
cout << "Join of string vector=" << string::join(vector6, "_") << endl;
cout << "Join of string_view vector=" << string::join(vector5, "_") << endl;
```

Example of implementation in [\[2\]](#)

3.2.2 join method

join method will use the current string as a separator to join the list, it's the way of usage of join in Python

```
template<class T>
basic_string<CharT, Traits, Allocator> join(T &InputStringList)
```

Example of usage

```
cout << "pythonic Join of string =" << "-".join(vector6) << endl;
```

This option could also be part of `std::basic_string_view` class, the static option make less sense.

Example of implementation in [\[2\]](#)

3.2.3 string_join function

`string_join` function act exactly as static function but is more adequate if `string_split` function is the selected option for split.

```
template<class T,class U>
basic_string<typename T::value_type::value_type, typename T::value_type::traits_type> string_join(T &InputStringList, U Separator)
```

Example of usage (simple but so useful)

```
cout << "Join of string_view vector with string_join=" << string_join(vector5, "_") << endl;
```

Example of implementation in [\[2\]](#)

3.2.4 Processing optimization

It's possible to optimize the processing by iterating on the container a first time to compute the size of the final string and reserve it below an example on

```
template<class T>
basic_string<typename T::value_type::value_type, typename T::value_type::traits_type>
    string_join(const T &InputStringList
        , const basic_string_view<typename T::value_type::value_type,typename T::value_type::traits_type> Separator)
{
    basic_string<T::value_type::value_type, T::value_type::traits_type> result_string;
    size_t StrLen = 0;
    if (InputStringList.empty())
        return result_string;
    auto it = InputStringList.begin();
    for (; it != InputStringList.end(); ++it)
        StrLen += it->size() + Separator.size();
    result_string.reserve(StrLen);
    result_string += *InputStringList.begin();
    for (it = ++InputStringList.begin(); it != InputStringList.end(); ++it)
    {
        result_string += Separator;
        result_string += *it;
    }
    return result_string;
}
```

However it implies to be able to obtain the length of each string in both `InputStringList` and `Separator`. We can consider it's quite common that the separator will be a `char*` but it doesn't have `size()` member, so as a workaround the separator is specified as `const string_view`.

The problem is the same if `InputStringList` is a `vector<char*>`, but in this case the problem is bigger as it seems to me impossible to specify the type of the returned string.

3.2.5 Parameter order

In several function like `string_join` it was highlighted the order could be with the separator first as it's smaller than the container (or the input string).

In some language the standard implementation is with the separator first : PHP , C# , Java

In some others, it's separator last Go, Rust, `boost::algorithm::join`, LibC `strtok`

Python has a different logic as the separator is the "caller" object (like join classic method described earlier)

Having the separator as 2nd parameters would allow have it optional with ""

It has to be analyzed later, but the consensus seems to be for the separator last.

3.3 Discussion on case management function

This chapter wasn't in the initial perimeter of the proposal, but as in fact this proposal talk about solutions to help to handle `std::string`, I have decided to add it in order to avoid managing a separate proposal.

3.3.1 tolower/toupper for string

Today STL provide solutions to convert in lowercase/uppercase a single char, but applying it to a string is quite non-intuitive when we can hope a modern language should handle that easily.

```
std::string result;
std::transform( src.begin(), src.end(), std::back_inserter( result ), ::tolower );
```

When it could be `std::string result=src.lower()`; The proposal is to add `tolower` and `toupper` member method `basic_string` and `basic_string_view` classes has shorthand for string of existing `tolower` and `toupper` function of `cctype` header which return a converted copy.

As discussed for `split` & `join` it could also be non-members functions `string_toupper` there are quite the same Pros & Cons

Possible implementation

```
basic_string<CharT, Traits> tolower() const
{
    basic_string<CharT, Traits> result;
    result.reserve(size()); // Allows reserve the space
    std::transform(begin(), end(), std::back_inserter(result), ::tolower);
    return result;
}
```

3.3.2 In place tolower/toupper for string

The previous § propose methods returning transformed copy of the input string. But in some case we don't need to keep the original then it makes sense to reuse the memory instead of allocating a new one. Then we propose to have an in place version of methods suffixed by `_inplace` (we found a similar approach in POCO library)(perhaps an native English speaker may suggest a more adequate suffix(emplace like `emplace_back?`)).

```
void toupper_inplace();
```

Example of implementation in [\[6\]](#)

3.3.3 Locale version of tolower/toupper

Perhaps it could make sense to have version parametrized by a locale which uses `toupper` for locale header. Perhaps this version could handle 1:n conversion like 'ß'=>'SS' which is currently not handled by the existing single char `toupper`. Perhaps it makes sense to have it as non-members functions part of `locale` header.

3.3.4 case insensitive compare

It may append to wish to compare 2 strings in a case insensitive manner, but it's not easy to do it in an efficient manner. A naive solution consists to convert both string in lowercase on to call `compare` on the result. However this option may cause 2 extra memory allocation, when perhaps it can be detected immediately that on first char 'a'!='b' We propose to add a `icompare` method similar to the existing `compare` but not case sensitive in both `basic_string` and `basic_string_view` classes.

```
int icompare( const basic_string& str ) const;
```

IV. Proposed Text

Addition to <string_view> header

add in class basic_string_view

```
vector<basic_string<CharT, Traits> > splits(const basic_string_view &Separator) const;
vector<basic_string<CharT, Traits> > splits(const typename basic_string_view::value_type Separator) const;
vector<basic_string<CharT, Traits> > splits(const basic_regex<CharT> &Separator) const;

vector<basic_string_view> splitsv(const basic_string_view &Separator) const;
vector<basic_string_view> splitsv(const typename basic_string_view::value_type Separator) const;
vector<basic_string_view> splitsv(const basic_regex<CharT> &Separator) const;

template <class F>
void splitf(const basic_string_view &Separator,F functor) const;
template <class F>
void splitf(const typename basic_string_view::value_type Separator,F functor) const;
template <class F>
void splitf(const basic_regex<CharT> &Separator,F functor) const;

template <class T>
void splitc(const basic_string_view &Separator,T &Result) const;
template <class T>
void splitc(const typename basic_string_view::value_type Separator,T &Result) const;
template <class T>
void splitc(const basic_regex<CharT> &Separator,T &Result) const;

basic_string<CharT, Traits> toupper() const;
void toupper_inplace() ;
basic_string<CharT, Traits> tolower() const;
void tolower_inplace();
int icompare( const basic_string_view& str ) const;
```

splits method and overloads

```
vector<basic_string<CharT, Traits> > splits(const basic_string_view &Separator) const
vector<basic_string<CharT, Traits> > splits(const typename basic_string_view::value_type Separator) const
vector<basic_string<CharT, Traits> > splits(const basic_regex<CharT> &Separator) const
```

Effects: split a string based on the separator and return the result in a vector of string

The separator can be :

- * a string
- * a single char
- * regexp

Returns: vector<string>

Remarks: if string is empty (size()==0) or single char==0 then the string will be split on every char

If the input string is an empty string "" then split methods return an empty container or never call the callback function.

splitsv method and overloads

```
vector<basic_string_view> splitsv(const basic_string_view &Separator) const
vector<basic_string_view> splitsv(const typename basic_string_view::value_type Separator) const
vector<basic_string_view> splitsv(const basic_regex<CharT> &Separator) const
```

Effects: split a string based on the separator and return the result in a vector of string_view

The separator can be :

- * a string
- * a single char
- * regexp

Returns: vector<string_view>

Remarks: if string is empty (size()==0) or single char==0 then the string will be split on every char

If the input string is an empty string "" then split methods return an empty container or never call the callback function.

splitf method and overloads

```
template <class F>
void splitf(const basic_string_view &Separator,F functor) const
template <class F>
void splitf(const typename basic_string_view::value_type Separator,F functor) const
template <class F>
void splitf(const basic_regex<CharT> &Separator,F functor) const
```

Effects: split a string based on the separator and call a unary function for each occurrence

The separator can be :

- * a string
- * a single char
- * regexp

Returns: void

Remarks: if string is empty (size()==0) or single char==0 then the string will be split on every char

If the input string is an empty string "" then split methods return an empty container or never call the callback function.

splitc method and overloads

```
template <class T>
void splitc(const basic_string_view &Separator,T &Result) const
template <class T>
void splitc(const typename basic_string_view::value_type Separator,T &Result) const
template <class T>
void splitc(const basic_regex<CharT> &Separator,T &Result) const
```

Effects: split a string based on the separator and return the result in the container passed as output parameter

The separator can be :

- * a string
- * a single char
- * regexp

Returns: void

Remarks: if string is empty (size()==0) or single char==0 then the string will be split on every char

If the input string is an empty string "" then split methods return an empty container or never call the callback function.

case management method

```
basic_string<CharT, Traits> toupper() const;
basic_string<CharT, Traits> tolower() const;
```

Effects: Return a copy of a string transformed in lowercase/uppercase.

Returns: the transformed copy

```
void toupper_inplace();
void tolower_inplace();
```

Effects: Transform the string in is lowercase/uppercase version.

Returns: void

Remarks: it replace the original string

```
int icompare( const basic_string_view& str ) const;
```

Effects: Make a comparison of string in a similar manner that compare but case insensitive

Returns: negative value if *this appears before the character sequence specified by the arguments, in lexicographical order

zero if both character sequences compare equivalent

positive value if *this appears after the character sequence specified by the arguments, in lexicographical order

Addition to <string> header

add in class basic_string

```
vector<basic_string> splits(const basic_string_view<CharT, Traits> &Separator) const;
vector<basic_string> splits(const typename basic_string_view<CharT, Traits>::value_type Separator) const;
vector<basic_string> splits(const basic_regex<CharT> &Separator) const;
vector<basic_string_view<CharT, Traits> > splitsv(const basic_string_view<CharT, Traits> &Separator) const;
vector<basic_string_view<CharT, Traits> > splitsv(const typename basic_string_view<CharT, Traits>::value_type Separator) const;
vector<basic_string_view<CharT, Traits> > splitsv(const basic_regex<CharT> &Separator) const;
template <class F>
void splitf(const basic_string_view<CharT, Traits> &Separator,F functor) const;
template <class F>
void splitf(const typename basic_string_view<CharT, Traits>::value_type Separator,F functor) const;
template <class F>
void splitf(const basic_regex<CharT> &Separator,F functor) const;
template <class T>
void splitc(const basic_string_view<CharT, Traits> &Separator,T &Result) const;
template <class T>
void splitc(const typename basic_string_view<CharT, Traits>::value_type Separator,T &Result) const;
template <class T>
void splitc(const basic_regex<CharT> &Separator,T &Result) const;
```

```
template<class T,class U>
static basic_string<CharT, Traits, Allocator> join(T &InputStringList, U Separator);
```

```
basic_string toupper() const;
void toupper_inplace() ;
basic_string tolower() const;
void tolower_inplace();
int icompare( const basic_string& str ) const;
```

splits,splitsv,splitf,splitc,tolower,toupper,icompare methods and overloads

Exactly the same behavior as their `basic_string_view` equivalent it's a shortcut.

join static method

```
template<class T,class U>
static basic_string<CharT, Traits, Allocator> join(T &InputStringList, U Separator)
```

Effects: return a string which join all string contained in the `InputStringList`. Add a separator between each string. If there's N string there's N-1 separator inserted.

Returns: Aggregated string

Remarks: if `InputStringList` is empty method returns an empty string.

VI References

- [1] Discussion on Google Groups https://groups.google.com/a/isocpp.org/forum/?utm_medium=email&utm_source=footer#!topic/std-proposals/JTKTThJ-7Ko
- [2] Example of implementation on GitHub with VC17 (testes also with GCC 6.2 MingW)
Test : https://github.com/laurent-n/cpp17_implode_explode/blob/master/testcpp17_1/testongcc.cpp
Lib : https://github.com/laurent-n/cpp17_implode_explode/blob/master/testcpp17_1/string17.h
- [3a] Implementation prototype done for N3593 Proposal
https://github.com/laurent-n/cpp17_implode_explode/blob/master/testcpp17_3/StringSplit.h
- [3b] Test code of implementation prototype done for N3593 Proposal
https://github.com/laurent-n/cpp17_implode_explode/blob/master/testcpp17_3/TestStringSplit.cpp
- [4] Example of implementation by range initiated by Nicol and amended by Laurent on GitHub
https://github.com/laurent-n/cpp17_implode_explode/blob/master/testcpp17_3/RangeSplitTest.cpp
- [5] N3593 Proposal
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3593.html>
- [6] Example of implementation of unified split version on GitHub with VC17 (testes also with GCC 6.2 MingW)
https://github.com/laurent-n/cpp17_implode_explode/tree/master/testcpp17_4

VII Acknowledgements

Thanks to Nicol Bolas, Thiago Macieira, Ville Voutilainen, Olaf van der Spek, Matthew Woehlke, Alexander Bolz, Marco Arena, Jakob Riedle for assistance on this paper.